

Artificial Intelligence for Medicine

Spring 2026

Indice

1	A Brief Introduction to Machine Learning	3
1.1	AI's Invisible Hand in Daily Life	4
1.2	AI's Invisible Hand in Medicine & Regulation	4
1.3	Introduction to Machine Learning	4
2	Supervised Learning: Definition and Model Evaluation	6
2.1	Machine Learning Taxonomy	6
2.2	Supervised Learning	6
2.3	Medical Examples Using the T/E/P Framework	7
2.4	Regression Performance Metrics	8
2.5	Classification Performance Metrics and ROC Analysis	8
2.6	Confusion Matrix in Depth	9
2.7	Metrics Derived from the Confusion Matrix	10
2.8	The ROC Plane	11
2.9	From Threshold Scores to the ROC Curve	12
3	Supervised Learning: Generalization	15
3.1	Linear Regression	15
3.2	Generalization	17
4	Regularization and Hyperparameter Optimization	19
4.1	Regularization	19
4.2	Hyperparameters and Validation Set	21

1 A Brief Introduction to Machine Learning

Historical timeline of AI

The development of AI can be traced through a series of peaks of enthusiasm and subsequent “winters” of reduced funding and interest:

- **1950** — Turing Paper: “*Can machines think?*” Alan Turing introduces the concept and the Turing Test.
- **1950s** — Information Theory era: theoretical foundations, prototyping.
- **1970s** — **First AI Winter**: overpromised results lead to funding cuts.
- **1980s** — **Expert Systems**: rule-based systems appear capable of human-like decision-making.
- **Mid-80s – Mid-90s** — **Second AI Winter**.
- **2010s** — **Deep Learning boom**: specialist pattern recognition through neural networks; the AlexNet 2012 breakthrough is a watershed moment.
- **2020s** — **The Causality Trap**: models predict well but cannot explain *why*; a potential Third AI Winter looms alongside rapid incremental progress and the open question of AGI.

Why AI boomed in the 2010s

Five concurrent drivers enabled the deep learning revolution:

1. **Increased computational power**: Growth of GPU and TPU capabilities; scalable cloud infrastructures.
2. **Big data explosion**: Vast datasets from social media, IoT, and online platforms.
3. **Advances in ML algorithms**: Rise of deep learning and neural networks (e.g., AlexNet, 2012).
4. **Open-source ecosystem**: Frameworks like TensorFlow and PyTorch democratised access to AI tools.
5. **Investment and industry adoption**: Significant capital from tech giants (Google, Amazon, ...).

Gartner Hype Cycle for AI

The Hype Cycle (Gartner) describes the maturity of technologies along five phases: *Innovation Trigger* → *Peak of Inflated Expectations* → *Trough of Disillusionment* → *Slope of Enlightenment* → *Plateau of Productivity*.

Notable observations across the 2023–2025 editions:

- **2023**: Generative AI and Foundation Models at the Peak; Computer Vision already on the Slope of Enlightenment (< 2 years to plateau).
- **2024**: AI Engineering at the Peak; Generative AI moves toward the Trough; Responsible AI and Prompt Engineering still rising.
- **2025**: AI Agents and Sovereign AI at the Peak; Generative AI and Edge AI descending; Model Distillation and Knowledge Graphs on the Slope.

The plateau timeline legend: ○ < 2 yrs, ● 2–5 yrs, ● 5–10 yrs, ▲ > 10 yrs, ⊗ obsolete before plateau.

“Publish or Perish” culture

The academic incentive structure rewards quantity of publications over rigorous, reproducible results. This is particularly relevant in AI/Medicine: inflated claims, poor validation on small datasets, and lack of clinical translation are common consequences. Students are expected to read AI papers critically with this bias in mind.

1.1 AI's Invisible Hand in Daily Life

AI systems operate silently in everyday applications; the course uses these examples to introduce the three fundamental task types:

- **Face ID** (smartphone unlock) — *pattern recognition*.
- **Streaming services** (Netflix, Spotify) — *prediction*: past viewing/listening history is used to recommend future content.
- **Traffic monitoring** — *detection*: vehicles localised and tracked in real time from camera feeds.
- **Parking occupancy detection** — *detection*: a cheaper alternative to sensor-based systems that require regular maintenance.
- **Road maintenance** (Robotiz3d / University of Liverpool) — autonomous detection of road surface defects to increase allocation efficiency and reduce accident risk.

1.2 AI's Invisible Hand in Medicine & Regulation

The critical question

Is it always acceptable for AI to act as an “invisible hand” in high-stakes domains such as medicine? Transparency and explainability become ethical and legal requirements, not optional extras.

European AI Act (EU Regulation)

The EU AI Act introduces a **risk-based pyramid** for regulating AI systems:

1. **Unacceptable risk** → **Prohibited AI practices**: systems that pose a clear threat to safety, livelihoods, and rights. *Examples*: government social scoring, voice-assistant toys encouraging dangerous behaviour.
2. **High risk** → **Regulated, subject to conformity assessment**:
 - Critical infrastructures (e.g., transport systems).
 - Safety components of products (e.g., AI in **robot-assisted surgery**).
 - Migration, asylum, border control (e.g., verification of travel documents).

Medical AI therefore falls predominantly in this category.

3. **Limited risk** → **Transparency obligations**: users must be informed they are interacting with a machine (e.g., chatbots) to allow an informed decision.
4. **Low and minimal risk** → **No obligations**: free use; examples include AI-enabled video games or spam filters.

1.3 Introduction to Machine Learning

Definitions: $AI \supset ML \supset Deep\ Learning$

- **Artificial Intelligence (AI)**: the theory and development of computer systems able to perform tasks *normally requiring human intelligence*.
- **Machine Learning (ML)**: a *subset* of AI techniques that use statistical learning to enable machines to **improve with experience** (data-driven rather than rule-driven).
- **Deep Learning (DL)**: a *subset* of ML that uses complex algorithms (artificial neural networks) to **learn from large amounts of data**, without hand-crafted feature extraction.

Arthur Samuel's definition (1958)

“Machine learning is the field of study that gives computers the ability to learn without being explicitly programmed.” — Arthur Samuel (1901–1990)

When explicit programming fails

Traditional algorithms require a programmer to encode every rule explicitly.

Algorithm 1 Find the Maximum of Two Values (explicit programming example)

Require: a, b

```

1: if  $a > b$  then
2:    $\max \leftarrow a$ 
3: else
4:    $\max \leftarrow b$ 
5: end if

```

Ensure: $\max$

However, for many real-world tasks **no explicit algorithm exists** because humans perform them unconsciously and cannot articulate the rules. **Face recognition** is the canonical example:

- Humans perform it unconsciously and are unable to explain how.
- No explicit algorithm can be written.
- A person's face is a *pattern*; a learning programme captures the pattern specific to that person from *data* (labelled images).

Traditional vs. ML approach

Traditional approach	Machine learning approach
Study the problem	Study the problem
Write an algorithm	Collect data
Evaluate	Train ML algorithm
Analyse errors $\rightarrow$ fix algorithm	Analyse errors $\rightarrow$ iterate
	Evaluate

The ML paradigm replaces hand-crafted rules with **data**: “*What we lack in knowledge, we make up for in data.*”

ML in a nutshell

Machine learning = **Algorithms** + **Data** working together to produce a model capable of extracting information (patterns, predictions, classifications) from new unseen data.

Data preparation: the dominant cost

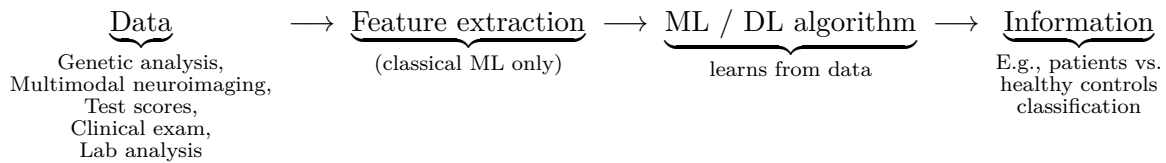
According to the *2020 State of Data Science* report (survey of $\approx 2,400$ professionals in >100 countries), the time breakdown of a typical ML project is:

Task	Time (%)
Data loading	19
Data cleansing	26
Data visualisation	21
Model selection	11
Model training and scoring	12
Deploying models	11
Data-related total	≈ 66

Historically, data prep has occupied 70–80% of a data scientist's time. This underscores that **data quality is the bottleneck**, not model sophistication.

The big picture: ML pipeline in medicine

A typical medical ML pipeline flows as follows:



Classical ML requires an explicit *feature extraction* step: domain experts design features (e.g., brain volume, cortical thickness) that are then fed to the algorithm.

Deep Learning eliminates the manual feature extraction step: the neural network *automatically learns* a hierarchy of representations directly from raw data (e.g., pixel intensities of an MRI volume). This is both a strength (less domain engineering) and a risk (black-box behaviour, harder to interpret clinically).

2 Supervised Learning: Definition and Model Evaluation

2.1 Machine Learning Taxonomy

Machine Learning is broadly divided into three paradigms:

- **Supervised Learning:** training data include desired outputs (labels). Subtypes: *Classification* (categorical label) and *Regression* (numeric label). Applications: image classification, diagnostics, customer retention, weather forecasting, advertising prediction.
- **Unsupervised Learning:** no labels provided; the algorithm finds hidden structure. Subtypes: *Clustering* (customer segmentation, targeted marketing), *Dimensionality Reduction* (feature elicitation, big data visualisation, meaningful compression), *Structure Discovery*. Applications: recommender systems.
- **Reinforcement Learning:** an agent learns by interacting with an environment and receiving reward signals. Applications: game AI, robot navigation, real-time decisions, skill acquisition, learning tasks.

2.2 Supervised Learning

Definition

In supervised learning, *the training data fed to the algorithm include the desired output, called labels*. The algorithm learns a mapping $f: \mathbf{x} \mapsto y$ from input features to output labels.

Typical task: predicting a future observation

- **Classification** — the label y is a *categorical variable* ($y \in \{1, \dots, C\}$). Medical example: benign vs. malignant lesion.
- **Regression** — the label y is a *numeric value* ($y \in \mathbb{R}$). Medical example: estimating cognitive decline based on MRI features.

Data representation

Given m subjects and n features:

- **Design matrix** $X_{(m \times n)}$: rows = subjects, columns = features. Row i is the feature vector $\mathbf{x}^{(i)} = (x_1^{(i)}, \dots, x_n^{(i)})$.

- **Output array $y_{(m)}$:** one label per subject.

Terminology note: Subject vs. Instance

- **Subject:** an individual in a medical/clinical dataset (patient, participant, case). Example: a patient with MRI, genetic, and clinical data.
- **Instance:** a general input-output pair used in ML, regardless of domain. Corresponds to a *row* in the dataset used for training or evaluation.

This distinction clarifies when referring to real-world medical data vs. abstract ML concepts.

Tom Mitchell’s formal definition (1997)

*“A computer program is said to learn from **experience** E with respect to some **task** T and some **performance measure** P , if its performance on T , as measured by P , improves with experience E .”*

The three components T , E , P provide a precise and operationalizable framework for any ML problem.

2.3 Medical Examples Using the T/E/P Framework

Example 1: Regression in Autism Spectrum Disorder (ASD)

- **Task (T):** Predict the ADOS score (Autism Diagnostic Observation Schedule — a quantitative measure of autism-related symptom severity).
- **Experience (E):** Genetic, neuroimaging (e.g., fMRI, sMRI), and clinical assessment data.
- **Performance metric (P):** MAE, MSE, RMSE (see below).

ADOS — Autism Diagnostic Observation Schedule

ADOS is a **semi-structured, standardised** assessment evaluating communication, social interaction, play, and restricted/repetitive behaviours in individuals suspected of having ASD. *Semi-structured* means the examiner follows a scripted set of activities but adapts the interaction naturally based on individual responses, combining consistency with flexibility. Activities include:

- Free play or construction tasks (e.g., building a toy).
- Asking about daily routines or emotions.
- Making a pretend phone call.
- Describing a picture or storytelling.

The assessment produces a **quantitative score** indicating symptom severity, used both in clinical diagnosis and as a *regression target variable* in ML.

Example 2: Classification in ASD

- **Task (T):** Classification — ASD vs. Typical Development.
- **Experience (E):** Genetic, neuroimaging, and clinical assessment data.
- **Performance metric (P):** Sensitivity, specificity, accuracy, AUC (area under the ROC curve).

2.4 Regression Performance Metrics

Given y_i the true value, $\hat{y}_i$ the predicted value, and m the number of samples:

MAE — Mean Absolute Error

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i| \quad (1)$$

Robust to outliers (uses absolute rather than squared differences). Expressed in the same units as y .

MSE — Mean Squared Error

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 \quad (2)$$

Penalises large errors more heavily (squared). Not in the same units as y .

RMSE — Root Mean Squared Error

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2} = \sqrt{\text{MSE}} \quad (3)$$

Restores the units of y ; more interpretable than MSE; still sensitive to outliers.

Regression metrics in scikit-learn

Scikit-learn uses **negative** error values (e.g., `neg_mean_squared_error`) to allow compatibility with score-maximisation logic (higher = better). Key scorers:

Scorer string	Function
'explained_variance'	<code>metrics.explained_variance_score</code>
'neg_mean_absolute_error'	<code>metrics.mean_absolute_error</code>
'neg_mean_squared_error'	<code>metrics.mean_squared_error</code>
'neg_root_mean_squared_error'	<code>metrics.mean_squared_error</code>
'neg_median_absolute_error'	<code>metrics.median_absolute_error</code>
'r2'	<code>metrics.r2_score</code>
'neg_mean_absolute_percentage_error'	<code>metrics.mean_absolute_percentage_error</code>

2.5 Classification Performance Metrics and ROC Analysis

ROC Analysis — Receiver Operating Characteristic

ROC analysis is a standard method for evaluating and comparing classification model performance. Key properties:

- Originally developed in *signal detection theory* to illustrate the trade-off between **true positive rate** (hit rate) and **false positive rate** (false alarm rate).
- Widely adopted in ML and medical diagnostics to analyse decision system behaviour.
- Addresses the limitations of relying *solely on accuracy*, especially in **imbalanced datasets** (where one class heavily dominates).

Dichotomous classifier — basic concepts

A **dichotomous classifier** maps each instance to one of two classes: $\{p, n\} = \{\text{positive}, \text{negative}\}$ (ground truth), predicted as $\{Y, N\}$. In the medical reference scenario:

- p = presence of disease; n = absence of disease.
- Y = classifier predicts disease present; N = classifier predicts healthy.
- Some classifiers output a **continuous score** (e.g., probability estimate) and require a **threshold** to assign a class label.
- Other classifiers output a **discrete class label** directly.

Confusion matrix (count view)

Given a dichotomous classifier applied to a test set, four outcomes are possible:

Actual \ Predicted	Positive (Y)	Negative (N)
Positive (p)	True Positive (TP)	False Negative (FN)
Negative (n)	False Positive (FP)	True Negative (TN)

- **TP**: sick patient correctly identified as sick.
- **FN**: sick patient missed (incorrectly classified as healthy). Also called *Type II error*.
- **FP**: healthy patient incorrectly flagged as sick. Also called *Type I error*.
- **TN**: healthy patient correctly identified as healthy.

Confusion matrix (probabilistic view)

Each cell of the confusion matrix can be expressed as a conditional probability (estimated from the test set as a relative frequency):

Actual \ Predicted	Positive (Y)	Negative (N)
Positive (p)	$P(Y p) = \text{TP rate}$	$P(N p) = \text{FN rate}$
Negative (n)	$P(Y n) = \text{FP rate}$	$P(N n) = \text{TN rate}$

The four class rates are defined as:

$$\text{TP rate} = \frac{\text{Positives correctly classified}}{\text{Total positives}} = \frac{TP}{TP + FN} \quad (4)$$

$$\text{FP rate} = \frac{\text{Negatives incorrectly classified}}{\text{Total negatives}} = \frac{FP}{FP + TN} \quad (5)$$

$$\text{TN rate} = \frac{\text{Negatives correctly classified}}{\text{Total negatives}} = \frac{TN}{FP + TN} \quad (6)$$

$$\text{FN rate} = \frac{\text{Positives incorrectly classified}}{\text{Total positives}} = \frac{FN}{TP + FN} \quad (7)$$

Note: $\text{TP rate} + \text{FN rate} = 1$ and $\text{TN rate} + \text{FP rate} = 1$. These rates reflect the behaviour of the classifier under *known* ground-truth labels. The TP rate is also called **sensitivity** (or *recall*); the TN rate is also called **specificity**.

2.6 Confusion Matrix in Depth

Worked example (threshold = 0.5)

Consider a test set with 6 positive and 4 negative instances with a soft classifier (score output). Using threshold = 0.5 (predict Y if score > 0.5):

Instances	True class	Predicted	Note
1–6	p	Y	Correctly called positive (TP)
7–8	n	Y	score > 0.5 but truly negative (FP)
9–10	n	N	Correctly called negative (TN)

Resulting rates:

- $TPR = 6/6 = 1.0$ (100%): all sick patients correctly identified.
- $TNR = 2/4 = 0.5$ (50%): only half of healthy patients correctly identified.
- $FPR = 2/4 = 0.5$ (50%): half of healthy patients incorrectly flagged as sick.
- $FNR = 0/6 = 0.0$ (0%): no sick patient missed.

Probabilistic constraints

The rows of the probabilistic confusion matrix sum to 1:

$$P(Y | p) + P(N | p) = 1, \quad P(Y | n) + P(N | n) = 1 \quad (8)$$

Key implication: only **two** of the four cells are independent. A **perfect classifier** achieves $P(Y | p) = 1$ and $P(Y | n) = 0$.

2.7 Metrics Derived from the Confusion Matrix

Sensitivity, Specificity, Precision

Sensitivity (= Recall = TPR):

$$TPR = \frac{TP}{TP + FN} \quad (9)$$

Proportion of actual positives correctly identified. Clinically critical: low sensitivity $\Rightarrow$ sick patients missed.

Specificity (= TNR):

$$TNR = \frac{TN}{TN + FP} \quad (10)$$

Proportion of actual negatives correctly identified. Low specificity $\Rightarrow$ too many healthy patients unnecessarily treated.

Precision (= Positive Predictive Value, PPV):

$$PPV = P(p | Y) = \frac{TP}{TP + FP} \quad (11)$$

Proportion of predicted positives that are actually positive. Note: this is a *post-test* probability; it depends on both the classifier and the disease prevalence in the population.

A good classifier should simultaneously achieve high sensitivity, specificity, and precision. In practice, sensitivity and specificity are in tension (improving one typically worsens the other at a given threshold).

Accuracy and why it fails on imbalanced data

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (12)$$

Accuracy is the fraction of all predictions that are correct. It is intuitive but **highly misleading on imbalanced datasets**. Canonical counter-example: if 95% of instances are negative, a trivial classifier that *always predicts “N”* achieves 95% accuracy — yet has $TPR = 0$, detecting no positive at all.

F1-score

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (13)$$

The F1-score is the **harmonic mean** of Precision and Recall. It is preferred over accuracy when classes are imbalanced because it requires *both* Precision and Recall to be high.

Why the harmonic mean? The harmonic mean collapses towards the smaller of the two values, strongly penalising any extreme imbalance between the metrics.

Numerical example: Precision = 1.0, Recall = 0.1:

$$\text{Arithmetic mean} = \frac{1.0 + 0.1}{2} = 0.55 \quad (\text{misleadingly suggests acceptable performance}) \quad (14)$$

$$F_1 = 2 \cdot \frac{1.0 \times 0.1}{1.0 + 0.1} = 0.18 \quad (\text{correctly exposes that recall is catastrophically low}) \quad (15)$$

2.8 The ROC Plane

ROC plane definition

Since the confusion matrix has only **two degrees of freedom**, a dichotomous classifier at a given threshold can be fully represented as a **single point** in the ROC (Receiver Operating Characteristic) plane:

- *x*-axis: **FPR** = $1 - \text{TNR} = P(Y | n)$
- *y*-axis: **TPR** = Sensitivity = $P(Y | p)$

Reference points:

Point (FPR, TPR)	Interpretation
(0, 0)	Always predicts “N”: no positive ever called.
(1, 1)	Always predicts “Y”: everything called positive.
(0, 1)	Perfect classifier.
(0.5, 0.5)	Random classifier (50% positive rate).

Interpreting positions in the ROC plane

A classifier is **better** than another if its ROC point lies **higher and to the left**.

- **Conservative classifiers** (lower-left region): predict positive only with strong evidence → few FPs but few TPs as well.
- **Liberal classifiers** (upper-right region): predict positive even with weak evidence → many TPs but also many FPs.
- In most clinical settings, the **left region** (low FPR) is especially important: false alarms are costly.

The random classifier and the diagonal

A random classifier that predicts positive with constant probability π achieves:

$$\text{TPR} = \pi, \quad \text{FPR} = \pi \quad \Rightarrow \quad \text{point } (\pi, \pi) \quad (16)$$

Varying $\pi \in [0, 1]$ traces the **main diagonal** from (0, 0) to (1, 1).

- Points **above** the diagonal: better-than-chance performance.
- Points **below** the diagonal: worse than random — rare; usually indicates systematic label inversion (simply flipping predictions would fix it).
- The diagonal represents **no discriminative power**.

2.9 From Threshold Scores to the ROC Curve

Hard vs. soft classifiers

- **Hard classifiers** (e.g., decision trees with crisp splits): output a discrete label $\{Y, N\}$ directly $\rightarrow$ a *single point* in the ROC plane.
- **Soft classifiers** (e.g., logistic regression, neural networks, SVMs with probability calibration): output a *continuous score* $s \in \mathbb{R}$ representing confidence in the positive class.

For soft classifiers, a decision threshold θ converts the score to a label:

$$\hat{y} = \begin{cases} Y & \text{if } s > \theta \\ N & \text{otherwise} \end{cases} \quad (17)$$

Each value of θ gives a *different* confusion matrix $\Rightarrow$ a different point in the ROC plane.

Building the ROC curve

By sweeping θ from $+\infty$ to $-\infty$:

1. $\theta = +\infty$: nothing is called positive $\rightarrow$ point $(0, 0)$.
2. As θ decreases, more instances exceed the threshold and are called positive. If the next instance crossing the threshold is truly positive: TPR increases (step up). If it is truly negative: FPR increases (step right).
3. $\theta = -\infty$: everything called positive $\rightarrow$ point $(1, 1)$.

The path traced in the ROC plane is the **ROC curve**.

Properties of the ROC curve

- A ROC curve from a **finite** test set is a **step function** (staircase): each step corresponds to one instance crossing the threshold.
- As the number of test instances $\rightarrow +\infty$, the curve converges to a **smooth, continuous curve**.
- The curve always runs from $(0, 0)$ to $(1, 1)$.
- A **good classifier** produces a curve bowed towards the upper-left corner.
- Decreasing θ moves the operating point from the lower-left (*conservative*) to the upper-right (*liberal*) of the plane.

Practical note from the worked example (20 instances, 10 positive + 10 negative, scores ranging from 0.9 to 0.1): instances are sorted by descending score; at each successive threshold the (FPR, TPR) pair is recalculated and plotted. Steps move *up* when a positive is encountered and *right* when a negative is encountered.

Generating an ROC Curve from a Dichotomous Classifier

If we want to generate an ROC curve from a dichotomous classifier (i.e., one that only outputs "Y" or "N"), we need to obtain a score rather than just a class label. A single class label yields a single point in the ROC plane, not a full curve.

Fortunately, many dichotomous classifiers can be converted into scoring classifiers by "looking inside":

1. For example, decision trees can expose the proportion of positive samples in a leaf.

2. k-NN classifiers can use the fraction of neighbors in the positive class. Rule-based systems may have internal confidence values.
3. These internal values can be used as scores to vary the threshold and generate the full ROC curve.

Why Generate Scores from Classifiers with Predicted Labels?

Some classifiers (e.g., logistic regression, neural networks) estimate the probability of class membership and then apply a decision rule:

If $p > 0.5$ then predict Y

However, **the probability estimates may not be perfectly calibrated**. Using a fixed threshold (e.g., 0.5) might not yield optimal performance. It is often beneficial to **explore alternative thresholds** and analyze the trade-offs between true positives and false positives. The ROC curve allows us to evaluate classifier behavior across the full range of thresholds.

Classifier Calibration

Calibration means that the predicted probabilities match the observed frequencies. **Example:** if a classifier assigns a probability of 0.7 to 100 instances, then about 70 of them should actually belong to the positive class. Calibration is important when:

- Decisions are made based on predicted probabilities, not just class label;
- We need to estimate risk or uncertainty reliably

ROC Curve Properties

ROC curves have a very desirable property: **they are insensitive to changes in class distribution**. If the proportion of positive and negative instances in the test set changes, the ROC curve remains the same.

Class distribution corresponds to the relationship between the two rows of the confusion matrix:

- First row: actual positives
- Second row: actual negatives

Metrics that use values from both rows, such as accuracy, are affected by class imbalance. As the class distribution changes, those metrics will also change, even if the classifier's behavior stays the same. ROC curves are based on: **TPR (from the first row) and FPR (from the second row)**. → They are independent of the class distribution.

ROC Curves: Example of Class Imbalance Effect

Example: a classifier with the following fixed performance:

- Sensitivity (True Positive Rate): **10%**
- Specificity (True Negative Rate): **90%**

Now observe what happens to the **accuracy** as the class distribution varies:

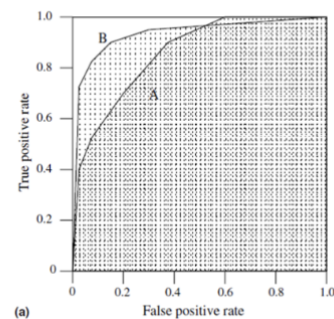
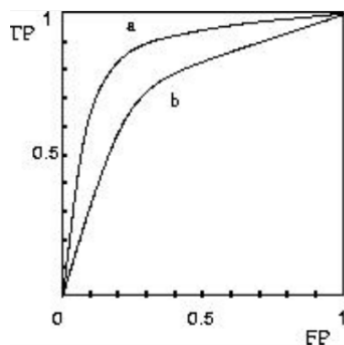
- **100 positive, 100 negative** → Accuracy = $(10 + 90)/(100 + 100) = 50\%$
- **100 positive, 1000 negative** → Accuracy = $(10 + 900)/(100 + 1000) = 83\%$
- **100 positive, 10,000 negative** → Accuracy = $(10 + 9000)/(100 + 10,000) = 89\%$

Conclusion: Accuracy increases with more negative cases, **even though the classifier's performance (TPR and TNR) stays the same.** This is why ROC curves are preferred: they are invariant to class distribution.

Area Under the ROC Curve (AUC)

- The ROC curve is a 2D plot that summarizes the performance of a classifier across all thresholds.
- To compare classifiers with a single number, we compute the **Area Under the ROC Curve (AUC)**.
- AUC ranges from 0 to 1:
 - $\text{AUC} = 0.5$ for a **random classifier**
 - $\text{AUC} > 0.5$ for any classifier with predictive value
 - $\text{AUC} = 1.0$ for a **perfect classifier**
- **Statistical interpretation:** AUC is the probability that the classifier assigns a higher score to a randomly chosen positive instance than to a randomly chosen negative one.
- AUC is threshold-independent and robust to class imbalance.

ROC Curve Comparison



- The **ideal ROC curve** connects the points $(0,0)$, $(0,1)$, and $(1,1)$, enclosing the maximum possible area ($\text{AUC} = 1$).
- If two ROC curves do **not intersect**, the one that lies consistently above the other represents the **better classifier** (i.e., higher AUC).
- If the ROC curves **intersect**, the comparison depends on the application's priorities:
 - Focus may be on regions with high specificity (low FPR) or high sensitivity (high TPR).
 - In such cases, the **partial AUC** within the relevant range is more informative.

Performance Metrics in scikit-learn — Classification Task

Scoring	Function	Comment
Classification		
'accuracy'	metrics.accuracy_score	
'balanced_accuracy'	metrics.balanced_accuracy_score	
'top_k_accuracy'	metrics.top_k_accuracy_score	
'average_precision'	metrics.average_precision_score	
'neg_brier_score'	metrics.brier_score_loss	
'f1'	metrics.f1_score	for binary targets
'f1_micro'	metrics.f1_score	micro-averaged
'f1_macro'	metrics.f1_score	macro-averaged
'f1_weighted'	metrics.f1_score	weighted average
'f1_samples'	metrics.f1_score	by multilabel sample
'neg_log_loss'	metrics.log_loss	requires predict_proba support
'precision' etc.	metrics.precision_score	suffixes apply as with 'f1'
'recall' etc.	metrics.recall_score	suffixes apply as with 'f1'
'jaccard' etc.	metrics.jaccard_score	suffixes apply as with 'f1'
'roc_auc'	metrics.roc_auc_score	
'roc_auc_ovr'	metrics.roc_auc_score	
'roc_auc_ovo'	metrics.roc_auc_score	
'roc_auc_ovr_weighted'	metrics.roc_auc_score	
'roc_auc_ovo_weighted'	metrics.roc_auc_score	

Overview of classification scoring options in scikit-learn's model selection tools

3 Supervised Learning: Generalization

3.1 Linear Regression

Supervised Learning: Linear Regression

In supervised learning with regression, the ML system receives as input a matrix X of dimensions $(m \times n)$ where:

- n : number of features
- m : number of subjects (samples)

The output is a column vector $\hat{y} \in \mathbb{R}^m$ of predicted numeric values.

Linear Regression (1 Feature, No Bias)

Linear regression with one input feature (no bias term, passes through the origin):

$$\hat{y} = w_1 x_1, \quad x_1 \in \mathbb{R}, w_1 \in \mathbb{R} \quad (18)$$

Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 \quad (19)$$

Linear Regression (2 and >2 Features, No Bias)

With two input features and no bias term:

$$\hat{y} = w_1 x_1 + w_2 x_2 \quad (20)$$

Vector notation:

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \in \mathbb{R}^2, \quad \mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} \in \mathbb{R}^2, \quad \hat{y} = \mathbf{w}^\top \mathbf{x} \quad (21)$$

General case with n features:

$$\mathbf{x} \in \mathbb{R}^n, \quad \mathbf{w} \in \mathbb{R}^n, \quad \hat{y} = \mathbf{w}^\top \mathbf{x} \quad (22)$$

Both $\mathbf{x}$ and $\mathbf{w}$ are column vectors. $\mathbf{w}$ is the vector of parameters (weights).

Linear Regression: Data Split

The dataset of m instances is split into:

- **Training set:** $X^{(\text{train})}$, $m^{(\text{train})}$ instances (rows)
- **Test set:** $X^{(\text{test})}$, $m^{(\text{test})}$ instances (rows)

where:

$$m^{(\text{train})} + m^{(\text{test})} = m \quad (23)$$

Linear Regression: Training

Training objective: minimize the MSE on the training set:

$$\text{MSE}_{\text{train}} = \frac{1}{m^{(\text{train})}} \sum_{i=1}^{m^{(\text{train})}} \left(y_i^{(\text{train})} - \hat{y}_i^{(\text{train})} \right)^2 = \frac{1}{m^{(\text{train})}} \sum_{i=1}^{m^{(\text{train})}} \left(y_i^{(\text{train})} - \mathbf{w}^\top \mathbf{x}_i \right)^2 \quad (24)$$

To minimize, set the gradient to zero:

$$\nabla_{\mathbf{w}} \text{MSE}_{\text{train}} = 0 \quad \Rightarrow \quad \text{Normal equations:} \quad X^{(\text{train})\top} X^{(\text{train})} \mathbf{w} = X^{(\text{train})\top} \mathbf{y}^{(\text{train})} \quad (25)$$

Explicit solution:

$$\mathbf{w} = \left(X^{(\text{train})\top} X^{(\text{train})} \right)^{-1} X^{(\text{train})\top} \mathbf{y}^{(\text{train})} \quad (26)$$

(Optional demonstration: Chapter 5 of *Deep Learning* by Ian Goodfellow et al.)

Why is it a Minimum? (1 Feature, No Bias)

The MSE loss is **convex** in w :

$$\text{MSE}_{\text{train}}(w) = \frac{1}{m} \sum_{i=1}^m (y_i - wx_i)^2 \quad (27)$$

It has the shape of a parabola. The **minimum** is found where the derivative is zero:

$$\frac{d}{dw} \text{MSE}_{\text{train}}(w) = 0 \quad (28)$$

This is the **global minimum** since the function is convex. Note: a function is convex if any line segment between two points on the graph lies above or on the graph itself.

Linear Regression with Intercept Term (Bias)

Adding a bias term leads to an affine transformation. One feature:

$$\hat{y} = w_1 x_1 + b, \quad x_1 \in \mathbb{R}, \quad w_1, b \in \mathbb{R} \quad (29)$$

Equivalent formulation: add a constant feature $x_2 = 1$:

$$\hat{y} = w_1 x_1 + b = w_1 x_1 + w_2 \cdot 1 = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}} \quad (30)$$

where:

$$\tilde{\mathbf{x}} = \begin{pmatrix} x_1 \\ 1 \end{pmatrix} \in \mathbb{R}^2, \quad \tilde{\mathbf{w}} = \begin{pmatrix} w_1 \\ b \end{pmatrix} \in \mathbb{R}^2 \quad (\text{bias as the last element of the vector}) \quad (31)$$

General case with n features: $\hat{y} = \mathbf{w}^\top \mathbf{x} + b = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$, with $\tilde{\mathbf{w}} \in \mathbb{R}^{n+1}$:

$$\tilde{\mathbf{x}} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \\ 1 \end{pmatrix} \in \mathbb{R}^{n+1}, \quad \tilde{\mathbf{w}} = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \\ b \end{pmatrix} \in \mathbb{R}^{n+1} \quad (32)$$

3.2 Generalization

Generalization and Test Error

The central challenge of machine learning: the algorithm must perform well on new, previously unseen inputs. This ability is called **generalization**.

The **generalization error**, also called **test error**, must be low:

$$\text{MSE}_{\text{test}} = \frac{1}{m^{(\text{test})}} \sum_{i=1}^{m^{(\text{test})}} \left(y_i^{(\text{test})} - \hat{y}_i^{(\text{test})} \right)^2 \quad (33)$$

Generalization: The Data-Generation Process

In supervised learning, we assume that both training and test data are generated from a probability distribution over input-output pairs (x, y) , called the **data-generation process**.

In simple terms: every time we build a new dataset, we get a different set of instances — but all are sampled from the same underlying distribution $P(x, y)$.

Clinical example: each time a new patient arrives in a hospital, their data (e.g., age, blood pressure, diagnosis, outcome) are generated by a complex combination of biological, environmental, and behavioral factors. This process is unknown and stochastic, but we assume that:

- Each patient instance is sampled independently
- Training and test sets come from the same distribution $P(x, y)$

Generalization: Data-Generation Process and i.i.d. Assumptions

We typically assume that data satisfy the **i.i.d. assumptions** (independent and identically distributed):

- **Independent:** each instance in the dataset is generated independently from the others
- **Identically distributed:** training and test instances are drawn from the same probability distribution

In other words: both sets contain the same type of data, with similar statistical properties (e.g., mean, variance, structure).

Note: if two instances represent data from the same patient at different time points, this violates the independence assumption due to temporal correlation.

If the test set comes from a different population (e.g., different hospital, machine, time period): the data are not identically distributed $\rightarrow$ this is known as **distribution shift**, a common real-world challenge.

Result: under the i.i.d. assumptions, the expected training error of a randomly selected model is equal to its expected test error.

Generalization: Training vs. Test Error — Underfitting and Overfitting

When using a ML algorithm, we do not fix the model parameters in advance and then sample data. Instead, we:

1. Sample a **training set**
2. Use it to choose the model parameters by minimizing training error
3. Then evaluate on a separately sampled **test set**

Important: under this process, the expected test error is **greater than or equal to** the expected training error.

A good algorithm should:

- Make the training error small
- Keep the gap between training and test errors small

Two central challenges in machine learning:

- **Underfitting:** training error remains high (poor fit to data)
- **Overfitting:** training error is low, but test error is much higher

Generalization: Model Capacity and Hypothesis Space

We can control whether a model is more likely to overfit or underfit by adjusting its **capacity**.

Model capacity: the ability of a model to fit a wide variety of functions.

- Low-capacity models may struggle to fit the training data → **underfitting**
- High-capacity models may memorize irrelevant patterns in the training data → **overfitting**

We control model capacity by choosing the **hypothesis space** of the learning algorithm.

Hypothesis space: the set of functions the algorithm is allowed to choose from.

Example:

- In linear regression, the hypothesis space contains all linear functions of the input
- We can increase capacity by extending the hypothesis space to include polynomial functions

Generalization: Polynomial Regression Models

Polynomial of degree 1 (linear regression):

$$\hat{y} = b + wx \quad (34)$$

Introducing an x^2 term (quadratic as a function of x):

$$\hat{y} = b + w_1x + w_2x^2 \quad (35)$$

Though quadratic in x , the output is still a **linear function of the parameters** → we can still use the normal equations to train in closed form.

Polynomial of degree 9:

$$\hat{y} = b + \sum_{i=1}^9 w_i x^i \quad (36)$$

Generalization: Underfitting and Overfitting

Changing the number of features and adding new parameters is a way to change a model's **capacity**. The error vs. capacity plot shows:

- **Underfitting zone:** both training error and generalization error are high

- **Optimal capacity:** minimum of the generalization error
- **Overfitting zone:** training error is low, but generalization error grows → **generalization gap**
- **Underfitting:** poor performance on training data and poor generalization to other data
- **Overfitting:** good performance on training data, poor generalization to other data

Occam's Razor and Model Selection

“Among competing models that explain the data equally well, the one with the fewest assumptions should be selected.” — William of Ockham (c. 1287–1347)

In machine learning, this is known as the **principle of parsimony**: prefer simpler models unless more complex ones provide significantly better generalization.

Why?

- Simpler models are less likely to overfit
- They tend to generalize better on unseen data
- They are easier to interpret and debug

Reminder: increasing model capacity without necessity can harm generalization.

4 Regularization and Hyperparameter Optimization

No Free Lunch Theorem (Wolpert, 1996)

No machine learning algorithm is universally any better than any other.

Key point:

- If one algorithm performs better than another on one class of problems, then it will perform worse on another class of problems.

A highly specialized algorithm may excel on a specific problem type but underperform on average across all problem types, while a general-purpose algorithm maintains consistent average performance.

4.1 Regularization

Regularization

The No Free Lunch Theorem implies that we must design machine learning algorithms to perform well on **specific tasks**. Two main strategies:

- **Modify the hypothesis space:** increase or decrease the model's capacity by adding or removing functions.
- **Introduce preferences within the hypothesis space:** favor some solutions over others, unless a less preferred solution fits the data significantly better.

Example: modify the training criterion in linear regression by adding **weight decay** (penalizing large weights).

Why Penalize Large Weights?

Penalizing large weights (e.g., via weight decay) improves generalization by:

1. Preventing overly complex models

- Large weights allow the model to fit the training data too closely, including noise and random fluctuations.
- This increases model complexity and leads to **overfitting**: the model performs well on training data but poorly on unseen data.
- **Example:** even a high-degree polynomial can be simple if its coefficients (weights) are small or close to zero.

2. Reducing model sensitivity

- Large weights amplify small variations in the input, making the model unstable.
- Smaller weights make the output less sensitive to small input perturbations.

3. Promoting more robust solutions

- Small weights ensure that no single feature dominates the prediction.
- The model relies on patterns that are more likely to persist across different datasets, improving robustness and generalization.

Large Weights vs. Small Weights

	Large weights	Small weights
Model complexity	High (risk of overfitting)	Moderate (better generalization)
Sensitivity to noise	High	Low
Robustness	Poor (unstable predictions)	Good (stable predictions)
Dependence on features	Strong dependence on few	Balanced reliance on multiple

Regularization pushes the model toward simpler, more robust solutions.

Linear Regression with Weight Decay

In weight-decayed linear regression, we minimize:

$$J(\mathbf{w}) = \text{MSE}_{\text{train}} + \lambda \mathbf{w}^\top \mathbf{w} \quad (37)$$

Interpretation: $J(\mathbf{w})$ expresses a preference for solutions with smaller squared L_2 norm of the weights.

Examples:

- Single feature: $\hat{y} = w_1 x_1 \Rightarrow J(w) = \text{MSE}_{\text{train}} + \lambda w_1^2$
- Two features: $\hat{y} = w_1 x_1 + w_2 x_2 \Rightarrow J(w) = \text{MSE}_{\text{train}} + \lambda(w_1^2 + w_2^2)$

Note: λ is a **hyperparameter** chosen in advance that controls the strength of regularization.

Effect of Weight Decay Strength

- When $\lambda = 0$: no preference for small weights $\rightarrow$ risk of overfitting.
- Increasing λ : forces the weights to become smaller $\rightarrow$ risk of underfitting.
- Minimizing $J(\mathbf{w})$ results in a **tradeoff** between fitting the training data and keeping the weights small.

The three regimes on a high-degree polynomial fit:

- **Underfitting** (excessive λ): model too constrained.
- **Appropriate weight decay** (medium λ): good generalization.
- **Overfitting** ($\lambda \rightarrow 0$): model memorizes training data.

Regularization: Expressing Preferences

Expressing preferences for one function over another is a more **general** way of controlling model capacity than simply including or excluding functions from the hypothesis space. There are many ways to express these preferences, collectively known as **regularization**.

Definition: regularization is any modification to a learning algorithm intended to **reduce generalization error** but not its training error.

4.2 Hyperparameters and Validation Set

Hyperparameters

Most machine learning algorithms have **hyperparameters** — settings that control the algorithm's behavior. Hyperparameters are **not learned** from the training data: they must be set manually or selected via a validation procedure.

Examples:

- Polynomial regression: the **degree** of the polynomial is a capacity hyperparameter.
- Weight decay: the regularization strength λ is a hyperparameter.

It is **not appropriate** to optimize hyperparameters that control model capacity using the training set: if optimized on the training set, hyperparameters would always choose maximum capacity, leading to overfitting. For instance, a higher-degree polynomial or setting $\lambda = 0$ always fits the training data better — but at the cost of poor generalization.

Solution: use a **validation set** — a separate set of examples not seen by the training algorithm — to select hyperparameters.

Hold-Out Validation Scheme

The dataset is randomly split into a training set and a test set. The test set is composed of examples drawn from the same distribution as the training set.

- Test instances must **never** be used to make choices about the model, including selecting hyperparameters.
- Simple hold-out validation is **not sufficient** for hyperparameter optimization: using the test set to tune hyperparameters leads to **optimistic bias** (data leakage).

Nested Hold-Out Validation Scheme

The dataset is randomly split into two disjoint parts:

- **Development set:** used for training the model and tuning hyperparameters. Further split into:
 - **Training set:** used to learn the model parameters.
 - **Validation set:** used to estimate generalization error and adjust hyperparameters.
- **Test set:** used only for the final evaluation.

This procedure ensures that hyperparameter optimization does **not bias** the test set evaluation.

Stratified Hold-Out and Nested Hold-Out

Stratified hold-out: instead of splitting the data randomly, we split while **preserving the class distribution** across training and test sets. Important especially with imbalanced datasets, ensuring both sets are representative of the original proportions.

Example: if the original dataset has 30% positives and 70% negatives, both training and test sets will maintain approximately the same 30/70 ratio.

Stratified nested hold-out: both the outer split and the inner split are performed in a stratified manner, preserving class balance during both hyperparameter tuning (inner split) and final evaluation (outer split).

Nested Hold-Out: Model Comparison and Feature Selection

Applications of nested hold-out beyond hyperparameter optimization:

- **Model comparison:** use the inner split to select the best model among several candidates (e.g., SVM, random forest, neural network); use the outer split to evaluate generalization of the selected model.
- **Feature selection:** use the inner split to select the best subset of features; use the outer split to estimate generalization error after feature selection.

Key idea:

- The **inner split** is used for making choices (hyperparameter tuning, model selection, feature selection).
- The **outer split** is used only for an unbiased evaluation of the final decision.

Validation Set Bias

Since the validation set is used to “train” the hyperparameters, its error tends to **underestimate** the true generalization error, although typically by less than the training error would. After hyperparameter tuning, the generalization error should be estimated **only on the test set**.

In practice: repeated reuse of the same test set over many years can lead to **optimistic bias** — test results no longer reflect true field performance. The community responds by moving to new and larger benchmark datasets.

Evolution of Benchmark Datasets

As benchmarks become saturated, new datasets are adopted to better evaluate true generalization (non-biomedical examples):

- **MNIST:** handwritten digits — early benchmark.
- **CIFAR-10/100:** small images.
- **ImageNet:** large-scale visual recognition.
- **Modern large datasets:** e.g., WIT, JFT, LAION.

These are examples from general computer vision, not from the biomedical field.